Scenario Controller — Script Reference & Smoke Test

Project: STING Bioinformatics · Unity 6000.4.9f1 · BNG VRIF **Scene used:** `Assets/Scenes/Hospital Room.unity` (work in a duplicate) **Purpose:** one page to understand every script, then run a 2-minute test that proves the system works.

---

1. The Scripts — one line each

Core (`Assets/Scripts/Scenario/Core/`)

Script	What it does
`ScenarioController.cs`	The engine. Walks the step list in order: enter a step, wait for it to report done, exit it, move to the next. Fires `onScenarioComplete` at the end.
`ScenarioData.cs`	The scenario asset. Just an ordered list of step assets you drag into place.
`ScenarioStepData.cs`	The base class every step asset inherits from. Holds data only.
`IScenarioStep.cs`	The runtime side of a step: `Enter()` and `Exit()`. Data and behaviour are kept separate on purpose.
`ScenarioContext.cs`	The shared toolbox handed to every step — the AudioSource, the Quiz, the UI root — plus `PlayVoice()`, the single audio path all voice-over goes through.

Event channels (`Assets/Scripts/Scenario/Events/`)

Script	What it does
`GameEvent.cs`	A signal with no data. Something calls `Raise()`, anyone listening reacts.
`GameEventGeneric.cs`	The generic base `GameEvent<T>` for channels that carry a value. Not used directly.
`StringGameEvent.cs`	A channel that carries a text id, so a step can wait for one *specific* task.
`SceneEventRelay.cs`	Listens to a `GameEvent` and fires a normal UnityEvent in the scene. Needed because an asset's UnityEvent cannot point at scene objects.
`TaskEventRaiser.cs`	Raises a `StringGameEvent` with its id. Right-click the component header → **Raise** to trigger it by hand while testing.

Step types (`Assets/Scripts/Scenario/Steps/`)

Script	What it does
`NarratorStepData.cs`	Plays an audio clip. Done when the clip ends. No clip = done instantly.
`InvokeSceneEventStepData.cs`	Raises a channel to make something happen in the scene. Done immediately, or waits for a completion channel if you tick that box.
`UIQuestionStepData.cs`	Shows a question on the Quiz UI, plays its prompt audio, checks the answer, plays correct/wrong feedback audio, then advances or re-asks.

Script	What it does
`WaitForTaskStepData.cs`	Pauses the scenario until a `StringGameEvent` is raised with a matching id.

Existing project scripts it uses

Script	What it does
`Quiz.cs`	Draws the question text and answer buttons. Raises `AnswerSelected` when a button is clicked.
`QuestionSO.cs`	A question asset: the text, the answers, which index is correct, and per-answer feedback.

---

2. Assets to create for the test

All under **`Assets ▸ Create ▸ Scenario`**.

Event channels ("EVs")

Asset	Type	Job in the test
`EV_ShowCube`	Events ▸ Game Event	Signal that means "reveal the cube".
`EV_ShowSphere`	Events ▸ Game Event	Signal that means "reveal the sphere".
`EV_TaskDone`	Events ▸ String Game Event	Signal that means "the player finished a task", carrying an id.

These are just empty mailboxes. They hold no logic — a step drops a message in, a relay picks it up.

Step assets

Asset	Type	Settings
`S1_ShowCube`	Steps ▸ Invoke Scene Event	*invokeChannel* = `EV_ShowCube`, *waitForExternalCompletion* off
`S2_WaitTask`	Steps ▸ Wait For Task	*taskChannel* = `EV_TaskDone`, *requiredTaskId* = `task1`
`S2b_Question`	Steps ▸ UI Question	*question* = `Question 1`, *questionVo* = `VO_QuestionPrompt`, *correctFeedbackVo* = `VO_Correct`, *wrongFeedbackVo* = `VO_Wrong`, *allowedTries* = `0`
`S3_ShowSphere`	Steps ▸ Invoke Scene Event	*invokeChannel* = `EV_ShowSphere`, *waitForExternalCompletion* off

Test audio lives in `Assets/Audio/_SmokeTest/` (short generated tones — delete once real narration exists).

The Scenario Data

Create **`SC_Smoke`** (Scenario ▸ Scenario Data) and drag the steps into *Steps* in this order:

```
1. S1_ShowCube     → cube appears, finishes instantly
2. S2_WaitTask     → PAUSES here, waiting for a signal
3. S2b_Question    → question panel + audio, waits for an answer
4. S3_ShowSphere   → sphere appears, scenario complete
```

Reordering this list is the whole point of the system: the sequence *is* the data, and no code changes.

---

3. Scene setup

Object	What to add	Notes
`TEST_Cube`	Cube at `(5.4, 1.3, -10.4)`, scale `0.25`	**Deactivate it.**
`TEST_Sphere`	Sphere at `(6.6, 1.3, -10.4)`, scale `0.25`	**Deactivate it.**
`SCENARIO_TEST`	`ScenarioController`, 2× `SceneEventRelay`, `TaskEventRaiser`, `AudioSource`	Keep active. Relays must live on an active object or they never hear their channel.
`QuestionPrefab`	Drag in `Assets/Prefabs/UI/QuestionPrefab.prefab`	**Deactivate the root**, but leave `QuizCanvas` inside it **active**.

Relay wiring — first relay: *channel* = `EV_ShowCube`, response → `TEST_Cube` → `GameObject.SetActive`, **checkbox ticked**. Second relay: *channel* = `EV_ShowSphere`, response → `TEST_Sphere` → `SetActive`, ticked. An unticked box means `SetActive(false)` and nothing ever appears.

ScenarioController — *scenario* = `SC_Smoke`, *playOnStart* ticked.

Context block — *voSource* = the AudioSource on `SCENARIO_TEST`; *quiz* = the `QuizCanvas` child; *pcUiRoot* = the `QuestionPrefab` **root**; leave the rest empty.

> The step only switches *pcUiRoot* on and off. Point it at the root, not at `QuizCanvas` — the full-screen blue `BackGroundImage` is a *sibling* of `QuizCanvas`, so toggling `QuizCanvas` leaves the blue covering the screen.

TaskEventRaiser — *channel* = `EV_TaskDone`, *taskId* = `task1` (must match `S2_WaitTask`).

Make the panel VR-friendly

The prefab ships as Screen Space Overlay, which fills your whole view. On the `QuestionPrefab` root's Canvas set **Render Mode = World Space**, then on its RectTransform set **Width 1000, Height 650, Scale 0.001**, Position `(5.97, 1.8, -10.75)`, Rotation `(0, -40.66, 0)`. Those are the same values the main menu already uses, so the panel lands in front of the player. Also **disable the `Game Manager` object** during testing, or it will re-show the main menu in that same spot.

---

4. Run it

1. Press **Play** → `TEST_Cube` appears immediately. The scenario is now parked on step 2.

2. Select `SCENARIO_TEST`, right-click the `TaskEventRaiser` header → **Raise** → the question panel appears **and its prompt audio starts at the same moment**.
3. The question is "*What dose should be administered for a 62kg adult with pneumonia?*" — `300mg` **is the correct answer.** Click it.
4. A short rising tone plays, the panel hides, `TEST_Sphere` appears, and the scenario completes.
5. Click a wrong answer instead → low buzz → the question comes back and the prompt replays. That is the retry policy.
6. Right-click the `ScenarioController` header → **Begin** to replay without leaving Play mode.

---

5. If it doesn't work

Symptom	Cause
`Assets ▸ Create ▸ Scenario` missing	A compile error somewhere in the project. Check the Console — the menu only registers when the assembly builds.
Screen is entirely blue	*pcUiRoot* is set to `QuizCanvas` instead of the `QuestionPrefab` root.
Cube never appears	Relay response checkbox unticked, *invokeChannel* empty, or the relay sits on an inactive object.
Sphere never appears after Raise	*taskId* and *requiredTaskId* don't match.
Answer buttons don't respond	Set the Canvas's *Event Camera* to the camera the XR rig renders through.
Long pause after answering	Feedback VO gates completion — the step waits for the clip to end. Use short clips.

STING Bioinformatics — Scenario Controller smoke test. Companion to `ScenarioController.md`.